PROJECT BASED EXPERIMENTALLEARNING

COURSE – NOSQL

PROJECT REPORT

TOPIC – STUDENT MANAGEMENT SYSTEM USING MONGODB



SUBMITTED TO

Mr.Harendra Singh

SUBMITTED BY

Kartikey Tiwari

BCA (DS& AI)

ROLL NO. – 1250258240 (40)

SCHOOL OF COMPUTER APPLICATIONS

BABU BANARASI DAS UNIVERSITY,LUCKNOW

ACADEMIC SESSION 2026–2027

INDEX

SR.NO.	SECTION	PAGE
1	Acknowledgement	3
2	Abstract	4
3	Introduction	5
4	Objectives	5
5	Methodology	6
6	Implementation Details	7
7	MongoDB Operations Demonstrated	8
8	Results	9
9	Conclusion	10
10	Future Scope	10
11	References	11

REPORT BASIS

This report follows the structure and presentation style of the supplied sample project report, while the technical content is based on the supplied Student Management System Using MongoDB project.

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled “Student Management System Using MongoDB”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

I also acknowledge the learning resources and tools used during the implementation of the project, which helped me practice MongoDB queries and understand document-based data management.

— MOHD ASHAR ABBAS

ABSTRACT

This report presents a Student Management System developed using MongoDB, a NoSQL document-oriented database. The project demonstrates how student information can be stored as documents and managed through MongoDB queries and operations.

The system uses a Students database and a students collection containing 50 student records. Each record includes fields such as roll number, name, course, age, and marks. The project demonstrates database creation, collection creation, insertion, retrieval, filtering, updating, deletion, sorting, limiting, course-based searching, and document counting.

Comparison operators such as $eq, $gt, $lt, $gte, and $lte and logical operators such as $and and $or are also demonstrated. The project therefore provides practical exposure to core NoSQL database operations using MongoDB.

KEY FEATURES

- Document-based student data storage using MongoDB.
- Creation of a database and collection.
- Insertion of 50 student documents.
- CRUD operations: create, read, update and delete.
- Comparison and logical query operators.
- Sorting, limiting, course-based searching and counting.
- Practical execution and observation of MongoDB query outputs.

INTRODUCTION

MongoDB is a NoSQL database management system that stores information in document form. Instead of organizing data only in fixed rows and columns, MongoDB stores records as flexible documents. This makes it suitable for applications where data needs to be stored and queried in a document-oriented structure.

In this project, a Student Management System has been created using MongoDB. The database contains information about 50 students, including roll number, name, course, age and marks. A series of MongoDB operations are performed on these records to demonstrate practical database management.

The supplied project also demonstrates how records can be filtered using comparison operators, combined using logical operators, modified using update operations, removed using delete operations, ordered using sort, restricted using limit, searched by course and counted.

OBJECTIVES

- To understand the concept of NoSQL databases.
- To learn the basic features and document model of MongoDB.
- To create a database and collection in MongoDB.
- To insert and manage 50 student records.
- To perform CRUD operations on student documents.
- To use comparison operators such as $eq, $gt, $lt, $gte and $lte.
- To perform logical AND and OR operations.
- To sort, limit, search and count documents.

PROJECT DATA MODEL

FIELD	EXAMPLE	PURPOSE
rollno	101	Unique student roll number
name	Aarav Sharma	Student name
course	Computer Science	Course/program of the student
age	24	Student age
marks	65	Marks obtained by the student

METHODOLOGY

The project follows a practical sequence of MongoDB operations. The database is first selected, followed by collection creation and insertion of the student dataset. The stored records are then queried and manipulated using MongoDB operators.

STEP	ACTIVITY	DESCRIPTION
1	Create Database	Create or switch to the Students database using use Students.
2	Create Collection	Create a students collection inside the Students database.
3	Insert Records	Insert 50 student documents containing roll number, name, course, age and marks.
4	Retrieve Data	Display the stored student documents using find().
5	Filter Data	Use comparison operators to retrieve records based on age.
6	Update Data	Use updateOne() and updateMany() to modify documents.
7	Delete Data	Use deleteOne() and deleteMany() to remove records.
8	Logical Queries	Use $and and $or to combine conditions.
9	Organize Results	Use sort() and limit() to control query results.
10	Search and Count	Search by course and count documents in the collection.

BASIC DATABASE SETUP

```
use Students
```

This creates or switches to the Students database.

```
db.createCollection("students")
```

This creates the students collection.

IMPLEMENTATION DETAILS

The implementation is based on the MongoDB shell/query syntax used in the supplied project. The student dataset contains 50 documents with roll number, name, course, age and marks.

INSERTING STUDENT RECORDS

```
db.students.insertMany([
 {rollno:101, name:"Aarav Sharma", course:"Computer Science", age:24, marks:65},
{rollno:102, name:"Priya Verma", course:"Electronics", age:26, marks:72}, {rollno:103,
name:"Rahul Singh", course:"Mechanical", age:25, marks:68},
 ...
 {rollno:150, name:"Jyoti Verma", course:"Mechanical", age:25, marks:71}
])
```

The supplied project records successful insertion of all 50 student documents.

DISPLAYING RECORDS

```
db.students.find()
```

The find() operation retrieves documents from the students collection. The supplied project shows the stored records and notes that the shell displays batches of documents.

COMPARISON OPERATORS

OPERATOR	EXAMPLE QUERY	PURPOSE
$eq	db.students.find({ age: { $eq: 25 } })	Age exactly equal to 25
$gt	db.students.find({ age: { $gt: 30 } })	Age greater than 30
$lt	db.students.find({ age: { $lt: 25 } })	Age less than 25
$gte	db.students.find({ age: { $gte: 30 } })	Age 30 or above
$lte	db.students.find({ age: { $lte: 25 } })	Age 25 or below

MONGODB OPERATIONS DEMONSTRATED

UPDATE ONE

```
db.students.updateOne({ rollno: 102 }, { $inc: { age: 1 } })
```

Increases Priya Verma's age by one year. The supplied output shows matchedCount: 1 and modifiedCount: 1.

UPDATE MANY

```
db.students.updateMany({ course: "Computer Science" }, { $inc: { marks: 5 } })
```

Increases marks by 5 for Computer Science students. The supplied output shows 14 documents matched and modified.

DELETE ONE

```
db.students.deleteOne({ rollno: 101 })
```

Deletes the single document for roll number 101.

DELETE MANY

```
db.students.deleteMany({ course: "Mechanical" })
```

Deletes all Mechanical-course documents; the supplied output shows 11 documents deleted.

AND

```
db.students.find({$and: [ {age:{$gte:25}},{course:"Electronics"} ] })
```

Returns students who are at least 25 years old and are in Electronics.

OR

```
db.students.find({$or: [ {age:{$lte:25}},{course:"Electrical" } ] })
```

Returns students aged 25 or below, or students in Electrical.

SORT

```
db.students.find().sort({ age: 1 }).limit(5)
```

Sorts records by age in ascending order; the project also demonstrates descending order with -1.

LIMIT

```
db.students.find().limit(3)
```

Displays only the first three remaining documents.

SEARCHBYCOURSE

```
db.students.find({ course: "Electronics" })
```

Finds students whose course is Electronics.

COUNT

```
db.students.countDocuments({})
```

Counts documents currently present in the collection; the supplied project reports a final result of 38.

RESULTS

The project successfully demonstrated the core NoSQL operations required for managing a student dataset in MongoDB. The original dataset contained 50 student documents, and the queries were executed to retrieve, filter, modify, delete and organize the records.

OPERATION	RESULT	OBSERVATION
Initial records inserted	50	50 student documents inserted successfully.
updateOne()	1 matched / 1 modified	Age of roll no. 102 increased by one.
updateMany()	14 matched / 14 modified	Marks increased by 5 for Computer Science students.
deleteOne()	1 deleted	Roll no. 101 removed.
deleteMany()	11 deleted	Mechanical-course documents removed.
Final count	38	Count reported by the supplied project after the deletion operations.

LEARNING OUTCOMES

- Understanding of the MongoDB document model and NoSQL approach.
- Practical use of database and collection creation commands.
- Hands-on experience with CRUD operations.
- Understanding of comparison and logical operators.
- Ability to sort, limit, search and count MongoDB documents.
- Understanding of how query results change after update and delete operations.

PROJECT RESULT

The supplied project demonstrates a complete practical workflow for managing a student dataset in MongoDB, from database creation and insertion through querying, updating, deletion and final document counting.

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data. Through this project, I learned how to insert, retrieve, update and delete documents. I also learned how to sort data in ascending and descending order, how comparison operators such as $eq, $gt, $lt, $gte and $lte work, and how logical operators such as $and and $or can be used to combine conditions.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.

FUTURE SCOPE

- Develop a web-based student management interface connected to MongoDB.
- Add authentication and role-based access for administrators, teachers and students.
- Add attendance, subject-wise marks and semester-wise performance modules.
- Add dashboards and charts for student performance analytics.
- Deploy the database using MongoDB Atlas and the application on a cloud platform.
- Add validation, indexing and more advanced aggregation queries for larger datasets.

REFERENCES

- MongoDB Documentation – MongoDB Query and CRUD concepts.
- MongoDB documentation on comparison and logical query operators.
- MongoDB documentation on update, delete, sort, limit and count operations.

THANK YOU!

StudentManagementSystemUsingMongoDB

MOHDASHARABBAS

BCA (DS & AI) | Roll No. 1250258270 (28)

Babu Banarasi Das University, Lucknow